# Modelling Software-based Systems

## Correctness by Construction with the Modelling Language Event-B using the Refinement

Master Informatique

Dominique Méry
Telecom Nancy,Université de Lorraine

13 octobre 2026

dominique.mery@loria.fr

# Summary


1. CM4
2. Correctness by Construction
3. Refinement of models
4. Example of the clock
5. Summary of the refinement
6. Summary

# Current Section Summary


1. CM4
2. Correctness by Construction
3. Refinement of models
4. Example of the clock
5. Summary of the refinement
6. Summary

# Current Section Summary


1. CM4
2. Correctness by Construction
3. Refinement of models
4. Example of the clock
5. Summary of the refinement
6. Summary

i

# Correctness by Construction

- Correctness by Construction is a method of building software -based systems with **demonstrable correctness** for security- and safety-critical applications.
- Correctness by Construction advocates a **step-wise refinement** process from specification to code using tools for checking and transforming models.
- Correctness by Construction is an approach to software/system construction
  - starting with an abstract model of the problem.
  - progressively adding details in a step-wise and checked fashion.
  - each step guarantees and proves the correctness of the new concrete model with respect to requirements

# The Cleanroom Method as CbC

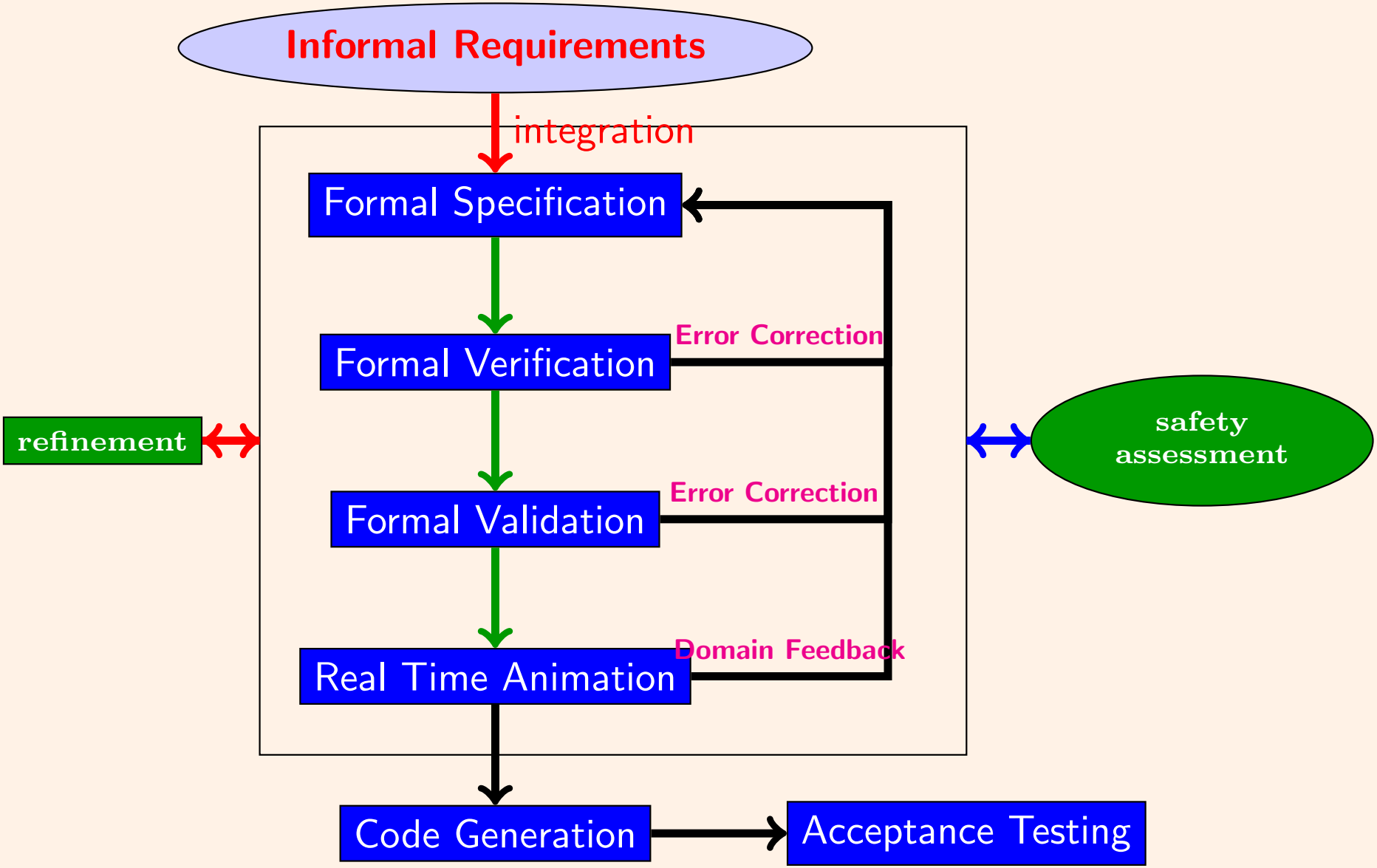
- The **Cleanroom** method, developed by Harlan Mills and his colleagues at IBM and elsewhere, attempts to do for software what cleanroom fabrication does for semiconductors : to achieve quality by keeping defects out during fabrication.
- In semiconductors, **dirt** or **dust** that is allowed to **contaminate** a chip as it is being made cannot possibly be removed later.
- But we try to do the equivalent when we write programs that are full of bugs, and then attempt to remove them all using debugging.

# The Cleanroom Method as CbC

The Cleanroom method, then, uses a number of techniques to develop software carefully, in a well-controlled way, so as to avoid or eliminate as many defects as possible before the software is ever executed. Elements of the method are :

- specification of all components of the software at all levels ;
- stepwise refinement using constructs called "box structures" ;
- verification of all components by the development team ;
- statistical quality control by independent certification testing ;
- no unit testing, no execution at all prior to certification testing.

# Critical System Development Life-Cycle Methodology



# Overview of Methodology

- Informal Requirements : Restricted form of natural language.
- Formal Specification : Modeling language like Event-B , Z, ASM, VDM, TLA+...
- Formal Verification : Theorem Prover Tools like PVS, Z3, SAT, SMT Solver...
- Formal Validation : Model Checker Tools like ProB, UPPAAL , SPIN, SMV ...
- Real-time Animation : **Our proposed approach ... Real-Time Animator ...**
- Code Generation : **Our proposed approach ... EB2ALL : EB2C, EB2C++, EB2J, EB2C# ...**
- Acceptance Testing : Failure Mode, Effects and Critically analysis(FMEA and FMEA), System Hazard Analyses(SHA)

# Case Studies

- *Colin Boyd and Anish Mathuria. Protocols Authentication and Key Establisment. Springer 2003.*
- *C. C. Marquezan and L. Z. Granville. Self-* and P2P for Network Management - Design Principles and Case Studies. Springer Briefs in Computer Science. Springer, 2012.*
- *Pacemaker Challenge Contribution*

# Current Section Summary


1. CM4
2. Correctness by Construction
3. Refinement of models
4. Example of the clock
5. Summary of the refinement
6. Summary

# Problems for Modelling systems

- Systems are generally very complex
- Invariant should be strong enough for proving safety properties
- Problems for modelling : finding suitable mathematical structures, listing events or actions of the system, proving proof obligations, ...

# Solution : refining models

- To understand more and more the system
- To distribute the complexity of the system
- To distribute the difficulties of the proof
- To improve explanations
- Validation (step by step)
- Refinement (invariant & behavior)

# refinement between two events (I)

## definition

Let $x$ be the abstract variable (or list of variables) and $I(s, c, x)$ the abstract invariant, $y$ the concrete variable (or list of variables) and $J(s, c, x, y)$ the concrete invariant.
Let c be a concrete event observing the variable $y$ and a an event observing the variable $x$ and preserving $I(s, c, x)$.
Event c refines event a with respect to $x$, $I(s, c, x)$, $y$ and $J(s, c, x, y)$, if

$$AX(s, c) \vdash I(s, c, x) \wedge J(s, c, x, y) \Rightarrow [c](\neg[a](\neg J(s, c, x, y)))$$

# Abstract event refined by a concrete event

$$a \left\{ \begin{array}{l} \textbf{ANY } u \textbf{ WHERE} \\ \quad G(u,s,c,x) \\ \textbf{THEN} \\ \quad x :\mid ABAP(u,s,c,x,x') \\ \textbf{END} \end{array} \right. \qquad c \left\{ \begin{array}{l} \textbf{ANY } v \textbf{ WHERE} \\ \quad H(v,s,c,y) \\ \textbf{WITNESS} \\ \quad u : WP(u,s,c,v,y) \\ \quad x' : WV(v,s,c,y',x') \\ \textbf{THEN} \\ \quad y :\mid CBAP(v,s,c,y,y') \\ \textbf{END} \end{array} \right.$$

The two events a and c are normalised by a relationship called BA(e)(s,c,x,x'), which simplifies the notations used.
The two events $a$ and $c$ are equivalent to events of the following normalized form :

- $a$ is equivalent to
  begin $x :\mid (\exists u.G(u,s,c,x) \wedge ABAP(u,s,c,x,x'))$ end
- $c$ is equivalent to
  begin $y :\mid (\exists v.H(v,s,c,y) \wedge CBAP(v,s,c,y,y'))$ end

# Explanations for the refinement

(Hypothesis)
(1) $AX(s,c) \vdash I(s,c,x) \wedge J(s,c,x,y) \Rightarrow [c](\neg[a](\neg J(s,c,x,y)))$
*equivalent to*
( Definition of $[a]$ : $[a](\neg J(s,c,x,y)) \equiv$
$\forall x'.(\exists u.G(u,s,c,x) \wedge ABAP(u,s,c,x,x')) \Rightarrow \neg J(s,c,x',y)))$
(2) $AX(s,c) \vdash I(s,c,x) \wedge J(s,c,x,y) \Rightarrow [c](\neg(\forall x'.(\exists u.G(u,s,c,x) \wedge$
$ABAP(u,s,c,x,x')) \Rightarrow \neg J(s,c,x',y))$
*equivalent to*
(Transformation by simplification of logical connectives)
(3) $AX(s,c) \vdash I(s,c,x) \wedge J(s,c,x,y) \Rightarrow [c](\exists x'.(\exists u.G(u,s,c,x) \wedge$
$ABAP(u,s,c,x,x')) \wedge J(s,c,x',y))$
*equivalent to*
( Definition of $[c]$)

(4) $AX(s,c) \vdash$
$I(s,c,x) \wedge J(s,c,x,y) \Rightarrow (\forall y'.(\exists v.H(v,s,c,x) \wedge CBAP(v,s,c,y,y')) \Rightarrow$
$((\exists x'.(\exists u.G(u,s,c,x) \wedge ABAP(u,s,c,x,x')) \wedge J(s,c,x',y')))$
*equivalent to*
(Transformation by quantifier elimination $\forall$)
(5) $AX(s,c) \vdash$
$I(s,c,x) \wedge J(s,c,x,y) \Rightarrow (\exists v.H(v,s,c,y) \wedge CBAP(v,s,c,y,y')) \Rightarrow$
$((\exists x'.(\exists u.G(u,s,c,x) \wedge ABAP(u,s,c,x,x')) \wedge J(s,c,x',y')))$
*equivalent to*
(Transformation by elimination of connector $\wedge$)
(6) $AX(s,c) \vdash$
$I(s,c,x) \wedge J(s,c,x,y) \wedge (\exists v.H(v,s,c,y) \wedge CBAP(v,s,c,y,y')) \Rightarrow$
$((\exists x'.(\exists u.G(u,s,c,x) \wedge ABAP(u,s,c,x,x')) \Rightarrow J(s,c,x',y')))$

*equivalent to*
(Transformation by elimination of quantifier $\exists$)
(7)
$AX(s,c) \vdash I(s,c,x) \wedge J(s,c,x,y) \wedge H(v,s,c,y) \wedge CBAP(v,s,c,y,y') \Rightarrow$
$((\exists x'.(\exists u.G(u,s,c,x) \wedge ABAP(u,s,c,x,x')) \wedge J(s,c,x',y')))$
*equivalent to*
(Transformation by property of quantifier $\exists$)
(8)
$AX(s,c) \vdash I(s,c,x) \wedge J(s,c,x,y) \wedge H(v,s,c,y) \wedge CBAP(v,s,c,y,y') \Rightarrow$
$((\exists x'.((\exists u.G(u,s,c,x) \wedge ABAP(u,s,c,x,x')) \wedge J(s,c,x',y')))$

*equivalent to*
(Transformation by elimination of $\wedge$)
(9)

1. $AX(s,c) \vdash I(s,c,x) \wedge J(s,c,x,y) \wedge H(v,s,c,y) \wedge CBAP(v,s,c,y,y') \Rightarrow (((\exists u.G(u,s,c,x))$
2. $AX(s,c) \vdash I(s,c,x) \wedge J(s,c,x,y) \wedge H(v,s,c,x) \wedge CBAP(v,s,c,y,y') \Rightarrow ((\exists x'.\exists u.(ABAP(u,s,c,x,x')) \wedge J(s,c,x',y')))$

## property refinement between events (II)

Let $x$ be the abstract variable (or list of variables) and $I(s, c, x)$ the abstract invariant, $y$ the concrete variable (or list of variables) and $J(s, c, x, y)$ the concrete invariant. the concrete invariant.
Let c be a concrete event observing the variable $y$ and a an event observing the variable $x$ and preserving $I(s, c, x)$.
Event c refines event a with respect to $x$, $I(s, c, x)$, $y$ and $J(s, c, x, y)$ *if, and only if,*

1. (GRD) $AX(s, c) \vdash I(s, c, x) \wedge J(s, c, x, y) \wedge H(v, s, c, x) \wedge CBAP(v, s, c, y, y') \Rightarrow \exists u.G(u, s, c, x)$
2. (SIM) $AX(s, c) \vdash I(s, c, x) \wedge J(s, c, x, y) \wedge H(v, s, c, x) \wedge CBAP(v, s, c, y, y') \Rightarrow ((\exists x'.\exists u.ABAP(u, s, c, x, x') \wedge J(s, c, x', y')))$

# Proof obligations for Event-B refinement)

- (INIT) $AX(s,c), CInit(s,c,y') \vdash \exists x'.(AInit(s,c,x') \wedge J(s,c,x',y')$
- (GRD)
  $AX(s,c), I(s,c,x), J(s,c,x,y), H(v,s,c,x), CBAP(v,s,c,y,y') \vdash$
  $(((\exists u.G(u,s,c,x))$
- (GRD-WIT)
  $AX(s,c), I(s,c,x), J(s,c,x,y), H(v,s,c,x), CBAP(v,s,c,y,y'),$
  $WP(u,s,c,v,y) \vdash G(u,s,c,x)$
- (SIM) $AX(s,c), I(s,c,x), J(s,c,x,y), H(v,s,c,x), CBAP(v,s,c,y,y') \vdash$
  $((\exists x'.(\exists u.ABAP(u,s,c,x,x')) \wedge J(s,c,x',y')))$
- (SIM-WIT)
  $AX(s,c), I(s,c,x), J(s,c,x,y), H(v,s,c,x), CBAP(v,s,c,y,y'),$
  $WP(u,s,c,v,y), WV(v,s,c,y,x') \vdash ABAP(u,s,c,x,x')) \wedge J(s,c,x',y')$
- (WFIS-P)
  $AX(s,c) \vdash I(s,c,x) \wedge J(s,c,x,y) \wedge H(v,s,c,x) \wedge CBAP(v,s,c,y,y') \vdash$
  $\exists u.WP(u,s,c,v,y)$
- (WFIS-V)
  $AX(s,c) \vdash I(s,c,x) \wedge J(s,c,x,y) \wedge H(v,s,c,x) \wedge CBAP(v,s,c,y,y') \vdash$
  $\exists x'.WV(v,s,c,y,x')$
- (TH) $AX(s,c) \vdash I(s,c,x) \wedge J(s,c,x,y) \vdash SAFE_1(s,c,x,y)$

**MACHINE** CM **REFINES** AM
**SEES** E
**VARIABLES** $y$
**INVARIANTS**
$jnv_1 : J_1(s, c, x, y)$
...
$jnv_r : J_r(s, c, x, y)$
**THEOREMS**
$th_1 : SAFE_1(s, c, x, y)$
...
$th_n : SAFE_n(s, c, x, y)$
**VARIANTS**
$var_1 : varexp_1(s, c, y)$
...
$var_t : varexp_t(s, c, y)$
**EVENTS**
**EVENT** initialisation
**BEGIN**
$y :| (CInit(s, c, y'))$
**END**
...
**EVENT** c **REFINES** $a$
**ANY** $v$ **WHERE**
$H(v, s, c, y)$
**WITNESS**
$u : WP(u, s, c, v, y)$
$x' : WV(v, s, c, y', x')$
**THEN**
$y :| CBAP(v, s, c, y, y')$
**END**
...
**END**

- The machine CM is a model describing a set of events E(CM ) modifying the $y$ variable declared in the clause **VARIABLES**.
- A clause **REFINES** indicates that the CM machine refines a AM machine and E(AM ) is the set of abstract events in AM .
- A particular event defines the initialisation of variable $y$ according to the relationship $CInit(s, c, y')$.
- The property " Event c refines event a with respect to $x$, $I(s, c, x)$, $y$ and $J(s, c, x, y)$" is denoted by the expression c refines a. Events a and c are attached to two machines AM and CM ; the invariant attached to each event is the invariant of its machine.

- A clause **INVARIANTS** describes the inductive invariant invariant $J(s, c, x, y)$ that this machine is assumed to respect provided that the associated verification conditions are shown to be valid in the theory induced by the context E mentioned by the clause **SEES**. $J(s, c, x, y)$ is the gluing invariant linking the variable $y$ to the variable $x$.
- The clause **THEOREMS** introduces the list of safety properties derived in the theory. These properties relate to the variables $y$ and $x$ and must be proved valid. It is possible to add theorems about sets and constants ; this can help the proofs to be carried out during the verification process.
- To conclude this description, we would like to add that events can carry very important information for the proof process, in particular for proposing witnesses during event refinement. Furthermore, each event has a status (ordinary, convergent, anticipated) which is important in the production of verification conditions. The clause **VARIANTS** is linked to events of convergent and anticipated status. The event c (concrete) explicitly refines an event a of the AM machine.

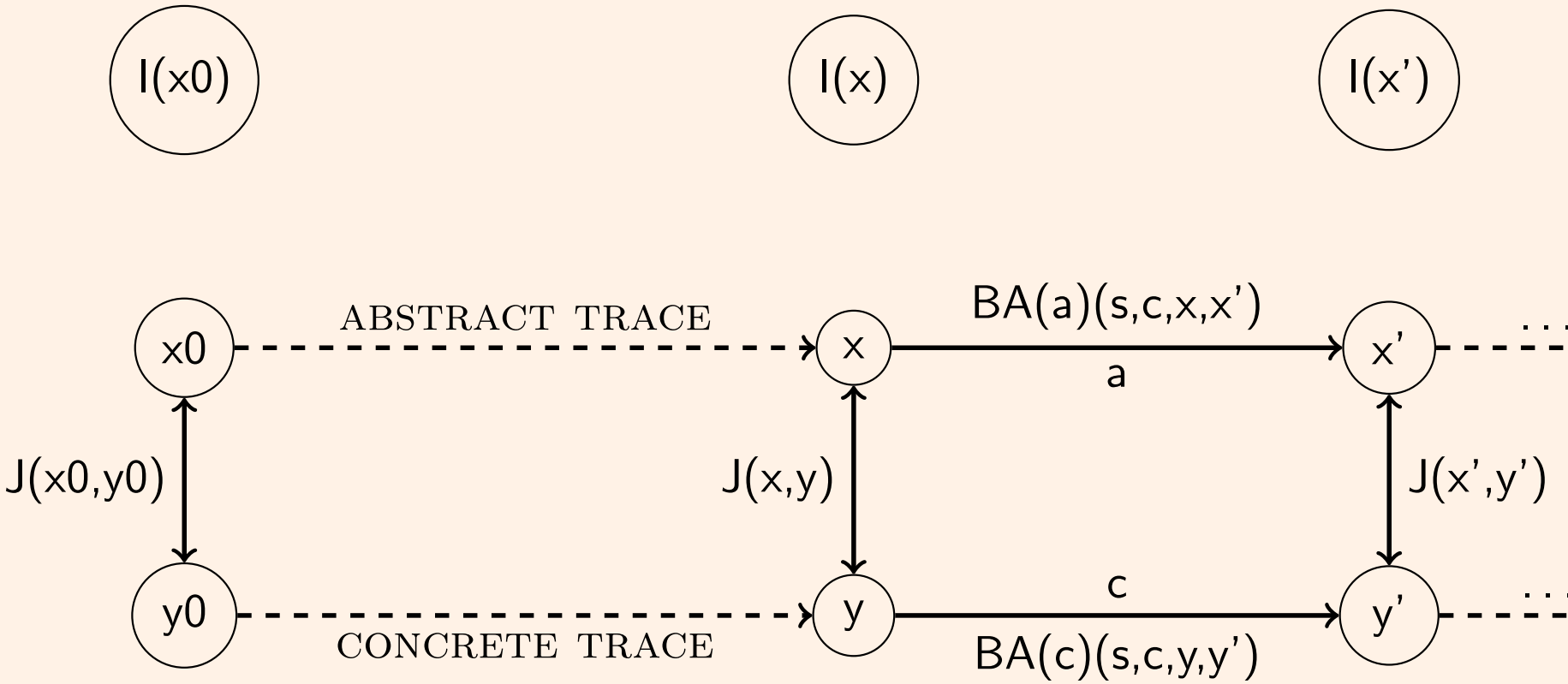
# Event-B machine refinement REFINES

**definition**

The machine CM refines the machine AM , if any event c of CM refines an event a of AM :
$\forall$c.c $\in$ E(CM ) $\Rightarrow$ $\exists$a.a $\in$ E(AM ) $\wedge$ e refines a.

- Each machine has an event skip which does not modify the machine's variables.
- A concrete event c can refine an event skip whose effect is not to modify $x$ in the abstract machine AM.
- The invariant of AM is $I(s, c, x)$ and that the initialisation of AM is $AInit(s, c, x')$.
- The proof witnesses are used to give properties of the parameter $u$ and the variable $x$ which have disappeared in the machine CM but for which the user must give an expression according to the state of CM .

# Refinement between two machines



# Current Section Summary


1. CM4
2. Correctness by Construction
3. Refinement of models
4. Example of the clock
5. Summary of the refinement
6. Summary

# Example of a clock

- A machine M0 models a device which provides the publication of the current time using the variable $h$.

# Example of a clock

- A machine M0 models a device which provides the publication of the current time using the variable $h$.
- A machine M1 models hours or a machine M1 reports observations of hours.

# Example of a clock

- A machine M0 models a device which provides the publication of the current time using the variable *h*.
- A machine M1 models hours or a machine M1 reports observations of hours.
- and a machine M2 reports hours and minutes.
- A very special case of refinement called *superposition* and the proof is fairly straightforward.

# Defining constants and sets

CONTEXT $C$

CONSTANTS $H$ $M$

AXIOMS
$@axm1\ H = 0..23$
$@axm2\ M = 0..59$
$end$

- set $H$ of hours is defined by an interval $0..23$.
- set $M$ of minutes is defined by an interval $0..59$

# M0 : A clock tells the time using a variable $h$

MACHINE $M0$ SEES $C$

VARIABLES $h$

INVARIANTS
$@inv1\ h \in H$

EVENTS
EVENT $INITIALISATION$
then
$@act1\ h :\in H$
end

EVENT $reading_some_hour$
then
$@act1\ h :|$
$((h < 23 \Rightarrow h' > h)$
$\wedge (h = 23 \Rightarrow h' \in 0..22) \wedge h' \in H)$
end

- set $H$ of hours is defined by an interval $0..23$.
- set $M$ of minutes is defined by an interval $0..59$

# M1 : A clock is progressing ...

MACHINE $M1$
REFINES $M0$ SEES $C$
VARIABLES $h$
INVARIANTS
$@inv1\ h \in H$
EVENTS
EVENT $INITIALISATION$
then
$@act1\ h :\in H$
end
EVENT $h1$
REFINES $reading_some_hour$
where
$@grd1\ h < 23$
then
$@act1\ h := h + 1$
end
EVENT $h2$
REFINES $reading_some_hour$
where
$@grd1\ h = 23$
then
$@act1\ h := 0$
end

- The variable $h$ is satisfying an invariant stating that $h \in H$.
- Two events are expresing the progress of $h$ and reach event refines $reading_some_hour$.

# M2 : A new variable $m$ records the minutes...

MACHINE $M2$
REFINES $M1$
SEES $C$

VARIABLES $h\ m$

INVARIANTS
$@inv1\ m \in M$
$theorem\ @inv2\ h \in H$

EVENTS
EVENT $INITIALISATION$
then
$@act1\ h :\in H$
$@act2\ m :\in M$
end

EVENT $h1m1$
where
$@grd1\ h < 23$
$@grd2\ m < 59$
then
$@act2\ m := m + 1$
end

- The variable $m$ is recording the minutes and satisfies the property $m \in M$.
- The property $h \in H$ is *inherited* from the previous machine M1 and is a theorem at the step.
- Event h1m1 is a new event modifyng onlly the variable $m$.

# M2 : A new variable $m$ records the minutes...

EVENT $h1m2$ REFINES $h1$
where
$@grd1\ h < 23$
$@grd2\ m = 59$
then
$@act1\ h := h + 1$
$@act2\ m := 0$
end

EVENT $h2m1$ REFINES $h2$
where
$@grd1\ h = 23$
$@grd2\ m = 59$
then
$@act1\ h := 0$
$@act2\ m := 0$
end

EVENT $h2m2$
where
$@grd1\ h = 23$
$@grd2\ m < 59$
then
$@act1\ m := m + 1$
end
$end$

- The new event h2m2 is modifying only $m$ but it is refining skip which is not explicitly mentioned.
- The two other events are reining events of M1 by modifying $h$ and $m$.

# Summary of the clock project

- Refinement improves the observation of the clock and the final machine M2 is validated by the animator/model-checker ProB.
- Only one interactive proof in the machibe M0 which is a do case process.

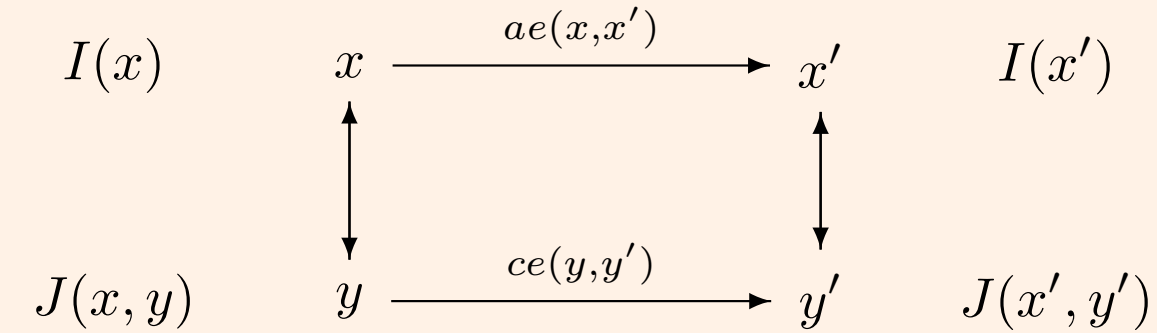
| Composant | Total | Auto | Inter | Rev | Und |
|---|---|---|---|---|---|
| mcfsi0-clock | 16 | 15 | 1 | 0 | 0 |
| C | 0 | 0 | 0 | 0 | 0 |
| M0 | 4 | 3 | 1 | 0 | 0 |
| M1 | 5 | 5 | 0 | 0 | 0 |
| M2 | 7 | 7 | 0 | 0 | 0 |

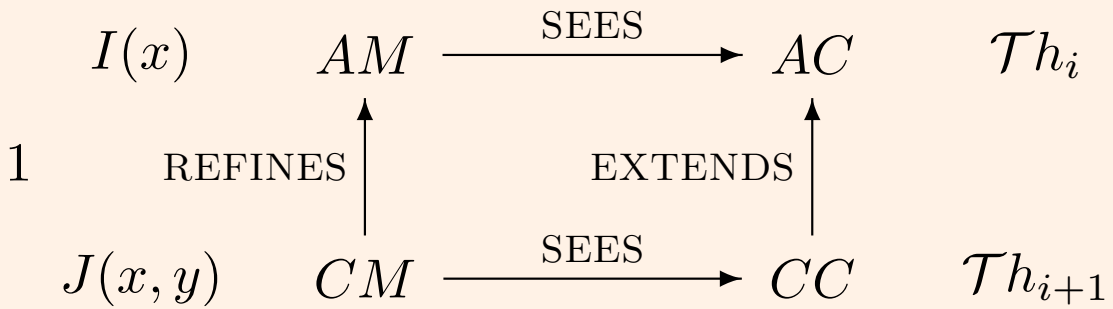
Table – PO summary for mcfsi0-clock

# Current Section Summary

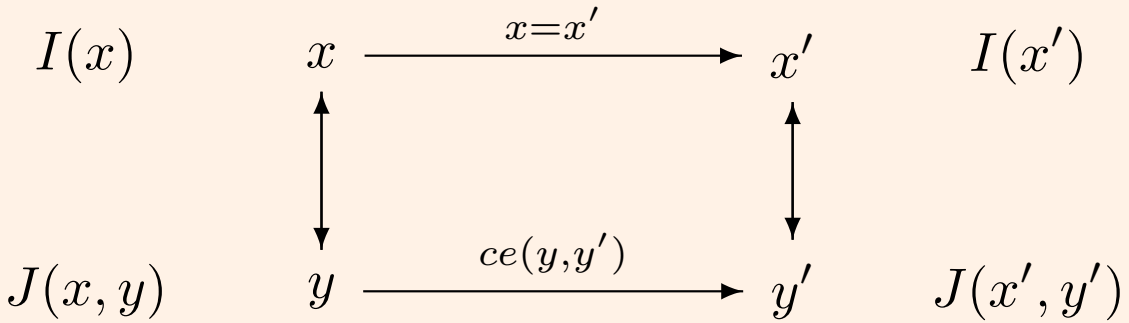

1. CM4
2. Correctness by Construction
3. Refinement of models
4. Example of the clock
5. Summary of the refinement
6. Summary

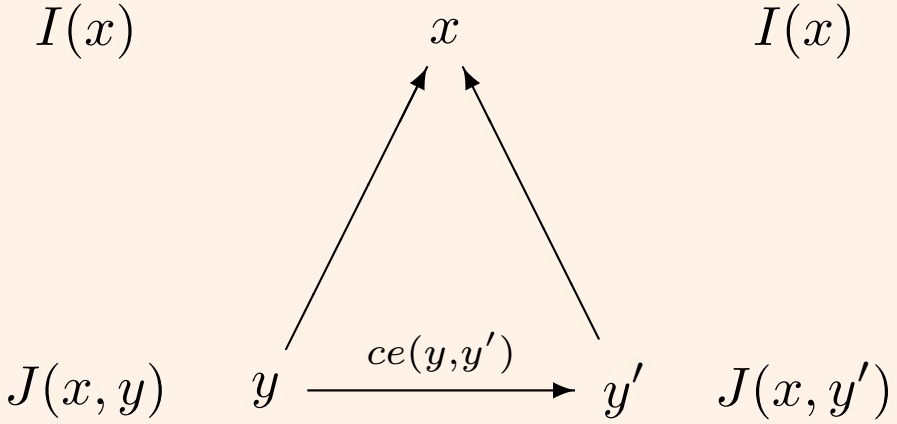
# Refinement of a model by another one (I)



# Refinement of a model by another one (II)



# Proof obligations for refinement

(REF1) : refinement of initial conditions

$$\textsc{InitC}(y) \Rightarrow \exists x * (\textsc{Init}(x) \wedge \mathrm{J}(x, y)) :$$

**The initial condition of the refinement model imply that there exists an abstract value in the abstract model such that that value satisfies the initial conditions of the abstract one and implies the new invariant of the refinement model.**

# Proof obligations for refinement

(REF2) : refinement of events

$$\mathrm{I}(x) \ \wedge \ \mathrm{J}(x,y) \ \wedge \ ce(y,y') \ \Rightarrow \ \exists x'.(ae(x,x') \ \wedge \ \mathrm{J}(x',y')) \ :$$

**The invariant in the refinement model is preserved by the refined event and the activation of the refined event triggers the corresponding abstract event.**

# Proof obligations for refinement

(REF3) : refinement of stuttering steps

$$\mathrm{I}(x) \quad \wedge \quad \mathrm{J}(x,y) \quad \wedge \quad ce(y,y') \;\Rightarrow\; \mathrm{J}(x,y') \quad :$$

**The invariant in the refinement model is preserved by the refined event but the event of the refinement model is a new event which was not visible in the abstract model ; the new event refines $skip$.**

# Proof obligations for refinement

(REF4) : Refinement does not introduce more blocking states

$$\mathrm{I}(x) \quad \wedge \quad \mathrm{J}(x,y) \quad \wedge \quad (G_1(x) \vee \ldots \vee G_n(x)) \quad \Rightarrow \quad H_1(y) \vee \ldots \vee H_k(y) \quad :$$

**The guards of events in the refinement model are strengthened and we have to prove that the refinement model is not more blocked than the abstract.**

# Proof obligations for refinement

(REF5) : Well-definedness of variant

$$\mathrm{I}(x) \quad \wedge \quad \mathrm{J}(x, y)) \quad \Rightarrow \quad V(y) \in \mathbb{N}$$

# Proof obligations for refinement

(REF6) : Well behaviour of new events

$$\mathrm{I}(x) \quad \wedge \quad \mathrm{J}(x,y) \quad \wedge \quad ce(y,y') \;\Rightarrow\; V(y') < V(y) \quad :$$

**New events should not block forever abstract ones.**

# Proof obligations for refinement

(REF7) : Feasibility of refined events

$$\Gamma(s,c) \vdash I(x) \wedge J(x,y) \wedge grd(E) \Rightarrow \exists y' \cdot P(y,y')$$

# Current Section Summary


1. CM4
2. Correctness by Construction
3. Refinement of models
4. Example of the clock
5. Summary of the refinement
6. Summary

# Final summary

- Refinement is powerful way to construct progressively a model.
- A model is then a sequence of successive machines related by refinement.
- Refinement means also simulation and preserves invariant of the abstraact machine.
- Refinement aims to implement the correctness by construction of systems.
- Refinement strategies are the key concepts of the success.
- Refinement patterns for :
  - design of sequential algorithms
  - design of recursive algorithms
  - design of distributed algorithms
  - design of reactive systems as pacemakers, hybrid systems, ...